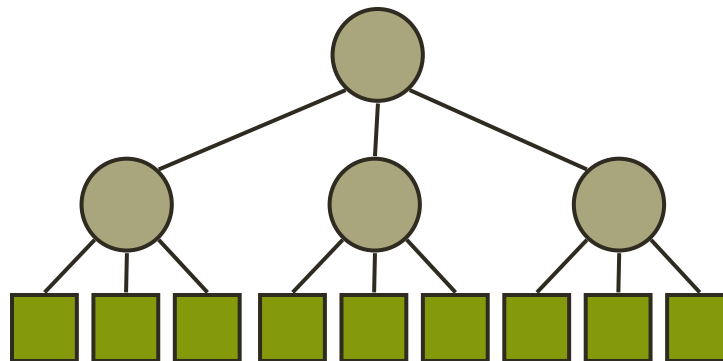


Advanced Algorithms and Data Structures

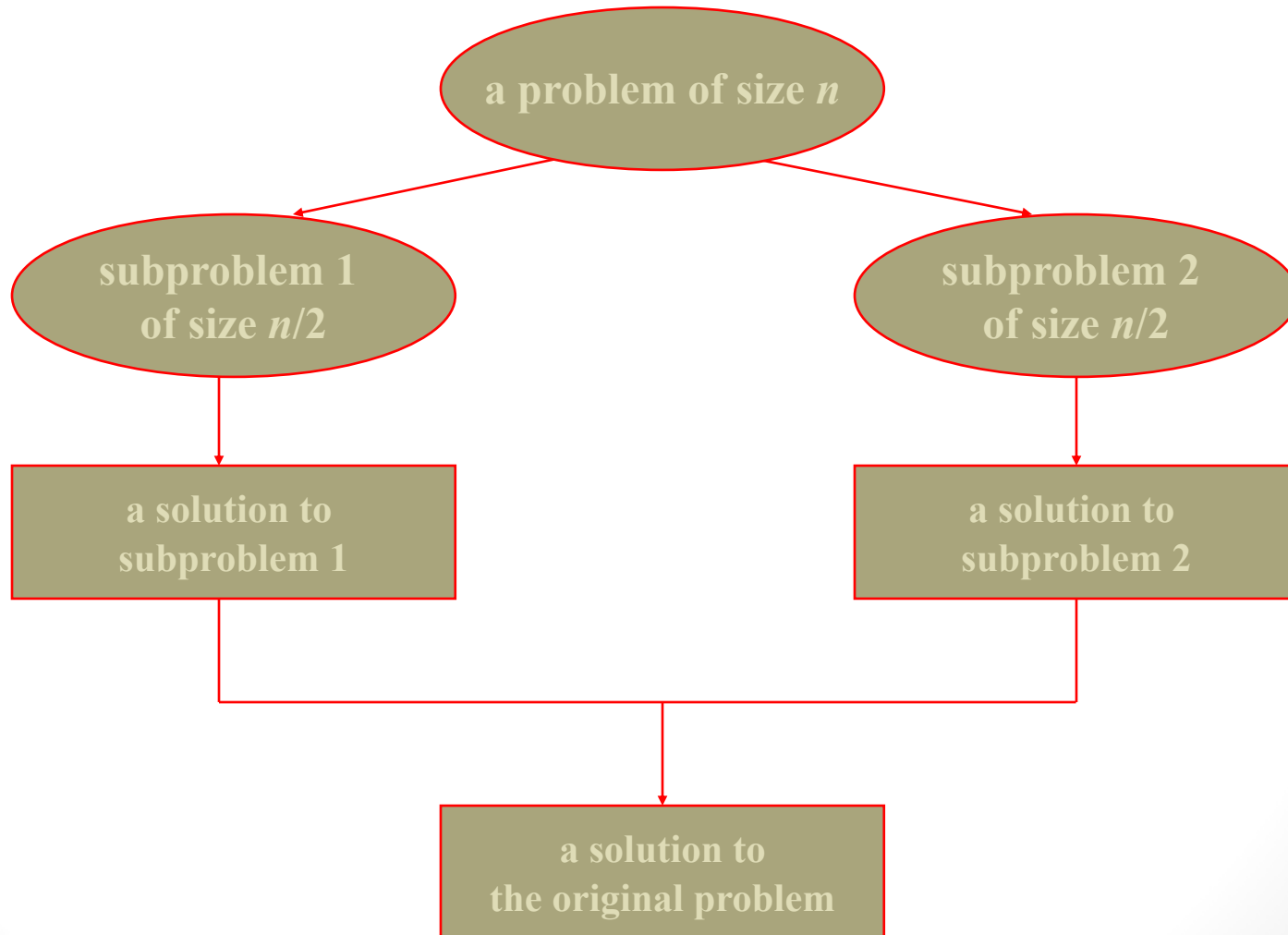
Lecture# 07: Divide and Conquer

Introduction

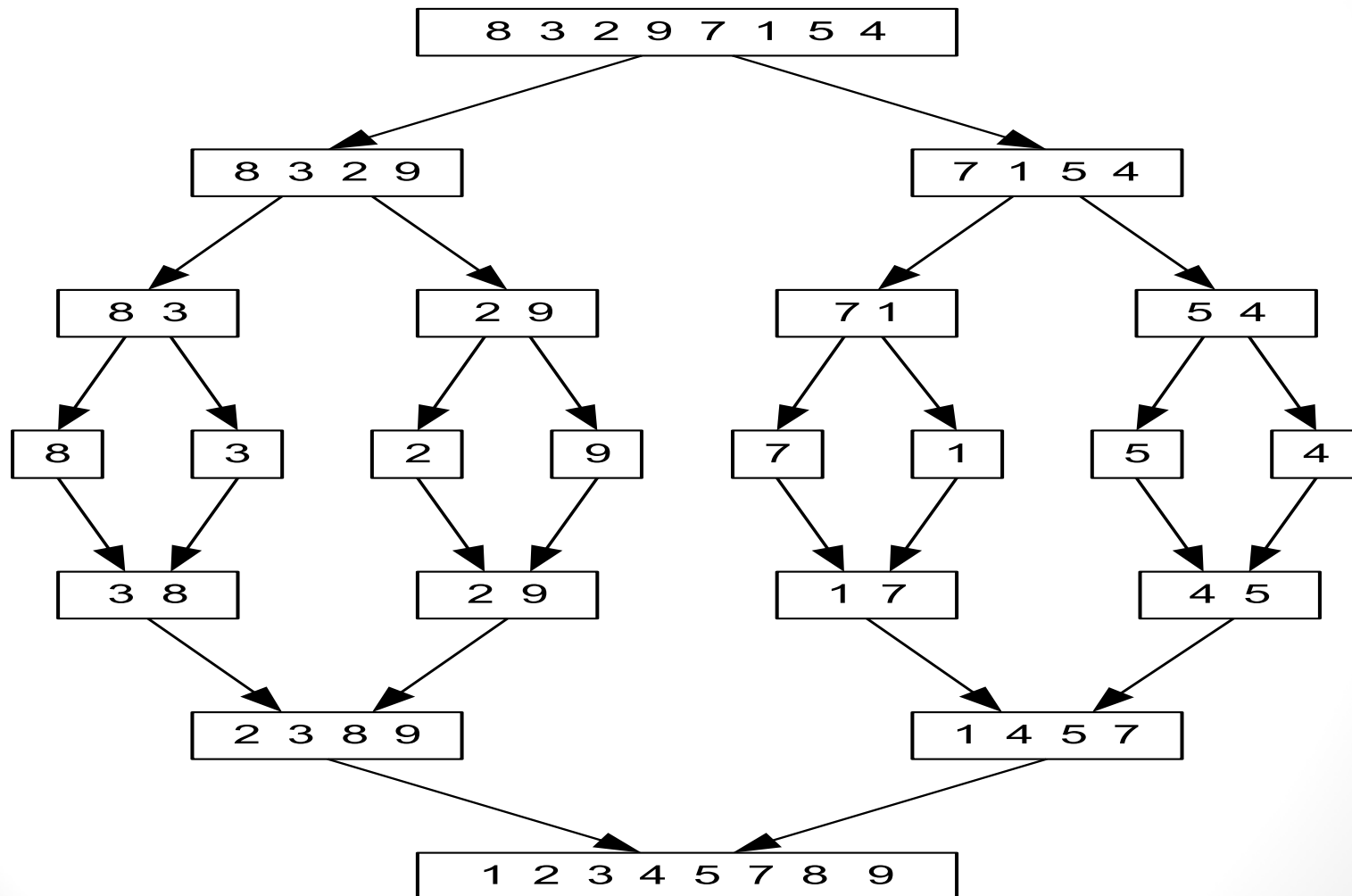
- Divide-and conquer is a general algorithm design paradigm:
 - Divide: divide the original problem S into smaller in two or more subproblems $S_1, S_2, \dots$ using recursion
 - Conquer: solve each of the smaller problems individually, and each subproblem should represent a part of the overall problem
 - Combine: combine the solutions for $S_1, S_2, \dots$ that are part of the recursive process to solve the actual problem S
- The base case for the recursion are subproblems of constant size
- Analysis can be done using **recurrence equations**



Divide and Conquer Technique



Example: Merge Sort



Merge Sort

- Split array $A[0..n-1]$ in two about equal halves and make copies of each half in arrays B and C
- Sort arrays B and C recursively
- Merge sorted arrays B and C into array A as follows:
 - ✓ Repeat the following until no elements remain in one of the arrays:
 - compare the first elements in the remaining unprocessed portions of the arrays
 - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
 - ✓ Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.

Merge Sort

- **Divide:** Given $A[p .. r]$, split the given array into two subarrays $A[p .. q]$ and $A[q+1 .. r]$ where q is the halfway point of $A[p .. r]$.
- **Conquer:** Recursively sort the two subarrays. If they are singletons, we have the base case.
- **Combine:** Merge the two sorted subarrays with a (linear) procedure Merge that iterates over the subarrays from the smallest element up to copy the next smallest element into a result array.

Pseudocode of Merge Sort

MERGE-SORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \lfloor (p + r) / 2 \rfloor$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q + 1, r$ )
5      MERGE( $A, p, q, r$ )
```

Pseudocode of Merge

MERGE(A, p, q, r)

```
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3  let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
```

Analysis of Merge

- The first two loops take $\Theta(n_1+n_2) = \Theta(n)$ time, where $n_1+n_2 = n$.
- The last loop makes n iterations, each taking constant time, for $\Theta(n)$ time.
- Total time will be $\Theta(n)$.

Analysis of D&C Algorithms

- **Recurrence equations** are used to describe the run time of Divide & Conquer algorithms. Let $T(n)$ be the running time on a problem of size n .
 - If n is below some constant (or often, $n=1$), we can solve the problem directly with brute force or trivially in $\Theta(1)$ time.
 - Otherwise we divide the problem into a subproblems, each $1/b$ size of the original. Often, as in Merge Sort, $a = b = 2$.
 - We pay cost $D(n)$ to divide the problems and $C(n)$ to combine the solutions.
 - We also pay cost $aT(n/b)$ solving subproblems.

Analysis of D&C Algorithms

- Then the total time to solve a problem of size n by dividing into a problems of size n/b can be expressed as:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise.} \end{cases}$$

Recurrence Analysis

- Merge-Sort is called with $p=1$ and $r=n$. For simplicity, assume that n is a power of 2. When $n \geq 2$, the time required is:
 - **Divide:** $\Theta(1)$ is required to compute q as the average of p and r .
 - **Conquer:** $2T(n/2)$ is required to recursively solve two subproblems, each of size $n/2$.
 - **Combine:** Merging an n -element subarray takes $\Theta(n)$ (this term absorbs the $\Theta(1)$ term for Divide).

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

[illegible]

Mathematical Recurrences

A recurrence is a function $T(n)$, that is defined in terms of its values with smaller inputs.

Example:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T\left(\frac{n}{2}\right) + n & n > 1 \end{cases}$$

Usually we drop the base case from the definition.

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

Mathematical Recurrences

Recurrences are used to represent the runtime of a recursive function

Cost of a recursive algorithm is equal to the cost of non-recursive part plus the recursive call on smaller input size.

We will be spending time looking at recurrences of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- a is the number of times a function calls itself
- b is the factor by which the input size is reduced
- $f(n)$ is the runtime of each function usually expressed in terms of Θ .

Recurrence Example

```
Factorial (n)
{
  if (n == 1)
    return 1;
  else
    return (n * Factorial(n-1));
}
```

The expression:

$$T(n) = \begin{cases} 1 & n = 1 \\ cT(n-1) + 1 & n > 1 \end{cases}$$

is a *recurrence*.

Solving Recurrences

Three methods are used in “solving” recurrences

- The Substitution Method
- The Recursion Tree Method
- The Master Method

Substitution Method

- In the iterative substitution, or “plug-and-chug,” technique, we iteratively apply the recurrence equation to itself and see if we can find a pattern: $T(n) = 2T(n/2) + bn$

$$= 2(2T(n/2^2)) + b(n/2) + bn$$

$$= 2^2 T(n/2^2) + 2bn$$

$$= 2^3 T(n/2^3) + 3bn$$

$$= 2^4 T(n/2^4) + 4bn$$

$$= \dots$$

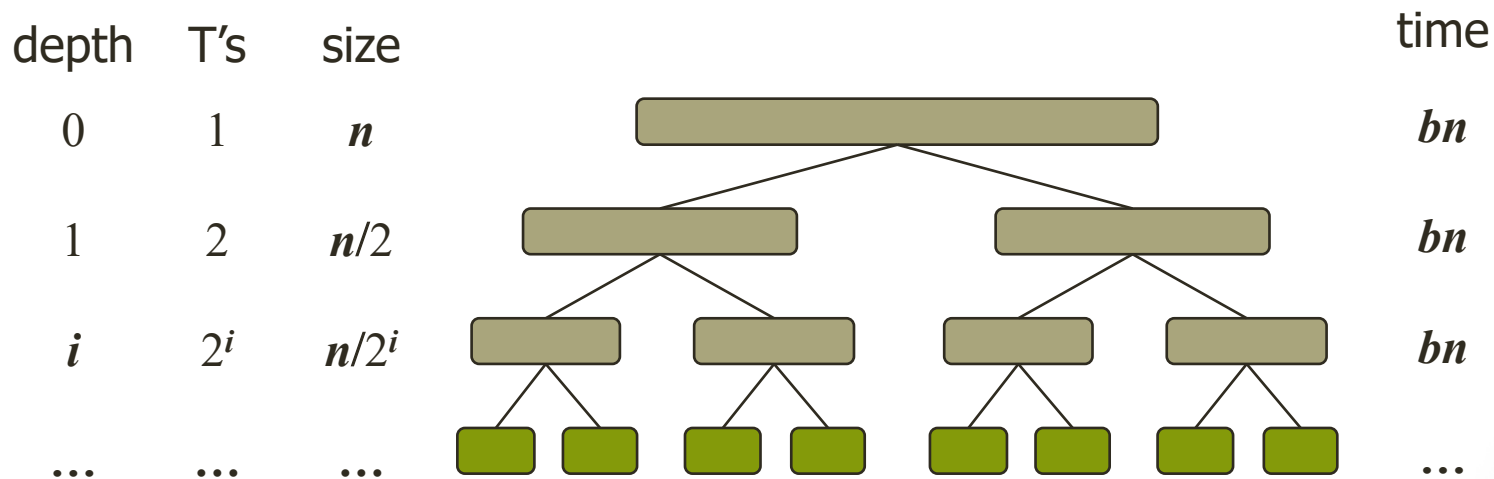
$$= 2^i T(n/2^i) + ibn$$

- Note that base, $T(n)=b$, case occurs when $2^i=n$. That is, $i = \log n$.
- So, $T(n) = bn + bn \log n$
- Thus, $T(n)$ is $O(n \log n)$.

The Recursion Tree

- Draw the recursion tree for the recurrence relation and look for a pattern:

$$T(n) = \begin{cases} b & \text{if } n < 2 \\ 2T(n/2) + bn & \text{if } n \geq 2 \end{cases}$$



Total time = $bn + bn \log n$
(last level plus all previous levels)

Master Method

The master theorem

The master method depends on the following theorem.

Theorem 4.1 (Master theorem)

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n),$$

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■